

Revision Sheet — Week 2: Basic Python Syntax

Your complete cheat sheet, self-test bank, and interview prep for variables, data types, operators, strings, and type conversion.

Key Syntax — Cheat Sheet

```
# — VARIABLES & DATA TYPES —————
name      = "Alice"      # str  - text in quotes
age       = 30           # int  - whole number
gpa       = 3.75         # float - decimal number
is_active = True         # bool  - True or False (capital!)

type(name)      # Returns <class 'str'>
isinstance(age, int) # Returns True - preferred type check

# — ARITHMETIC OPERATORS —————
a, b = 17, 5
a + b  # 22  - addition
a - b  # 12  - subtraction
a * b  # 85  - multiplication
a / b  # 3.4 - true division (always float)
a // b # 3   - floor division (rounds down)
a % b  # 2   - modulo (remainder)
a ** 2 # 289 - exponentiation

# — STRING OPERATIONS —————
s = "Hello, Python!"

len(s)      # 14      - number of characters
s[0]        # "H"     - indexing (0-based)
s[-1]       # "!"     - last character
s[0:5]      # "Hello" - slicing [start:stop] (stop excluded)
s[7:]       # "Python!" - slice to end
s[::-1]     # reversed string

s + " World" # concatenation
"-" * 20     # "-----" - repetition

s.upper()    # "HELLO, PYTHON!"
s.lower()    # "hello, python!"
s.strip()    # removes leading/trailing whitespace
s.replace("Python", "World") # "Hello, World!"
s.split()    # ['Hello,', 'Python!'] - splits on whitespace
s.split(",") # ['Hello', ' Python!'] - splits on comma
s.capitalize() # "Hello, python!" - first char upper, rest lower

# — TYPE CONVERSION —————
int("42")    # 42      - string to integer
float("3.14") # 3.14    - string to float
str(100)     # "100"   # number to string
int(3.9)     # 3       - float to int (truncates, does NOT round)
bool(0)      # False   - 0 is falsy; any other number is True
bool("")     # False   - empty string is falsy

# input() ALWAYS returns str - convert before arithmetic:
# age = int(input("Enter age: "))
```

Important Functions Table

Function	Syntax	What It Does
<code>print()</code>	<code>print(value, ...)</code>	Outputs values to the console; multiple args separated by space
<code>type()</code>	<code>type(x)</code>	Returns the data type class of <code>x</code>
<code>isinstance()</code>	<code>isinstance(x, type)</code>	Returns <code>True</code> if <code>x</code> is an instance of <code>type</code>
<code>len()</code>	<code>len(s)</code>	Returns the number of items/characters in a sequence
<code>int()</code>	<code>int(x)</code>	Converts <code>x</code> to integer (truncates floats, parses numeric strings)
<code>float()</code>	<code>float(x)</code>	Converts <code>x</code> to float
<code>str()</code>	<code>str(x)</code>	Converts <code>x</code> to its string representation
<code>bool()</code>	<code>bool(x)</code>	Converts <code>x</code> to boolean
<code>round()</code>	<code>round(x, n)</code>	Rounds <code>x</code> to <code>n</code> decimal places
<code>input()</code>	<code>input(prompt)</code>	Reads a line of user input; always returns <code>str</code>
<code>.upper()</code>	<code>s.upper()</code>	Returns uppercase copy of string <code>s</code>
<code>.lower()</code>	<code>s.lower()</code>	Returns lowercase copy of string <code>s</code>
<code>.strip()</code>	<code>s.strip()</code>	Returns copy with leading/trailing whitespace removed
<code>.replace()</code>	<code>s.replace(old, new)</code>	Returns copy with all occurrences of <code>old</code> replaced by <code>new</code>
<code>.split()</code>	<code>s.split(sep)</code>	Splits string into a list; splits on whitespace if no <code>sep</code> given
<code>.capitalize()</code>	<code>s.capitalize()</code>	First character uppercase, rest lowercase

Multiple Choice Questions

1. What is the output of `print(type(3.14))` ?

- A) `float`
- B) `<type 'float'>`
- C) `<class 'float'>`
- D) `decimal`

2. What does `17 // 5` evaluate to?

- A) `3.4`
- B) `3`
- C) `4`
- D) `2`

3. What is the output of `"ha" * 3` ?

- A) `"ha ha ha"`
- B) `"ha3"`
- C) `"hahaha"`
- D) `TypeError`

4. What does `"Python"[-1]` return?

- A) `"P"`
- B) `"Pytho"`

- C) `IndexError`
- D) `"n"`

5. Which of the following will cause a `TypeError` ?

- A) `"age: " + str(25)`
- B) `len("hello")`
- C) `"score: " + 95`
- D) `int("42") + 8`

6. What is the result of `int(float("3.99"))` ?

- A) 4 (rounds up)
- B) 3.99
- C) 3 (truncates)
- D) `ValueError`

7. What does `" hello ".strip().upper()` return?

- A) `" HELLO "`
- B) `"Hello"`
- C) `"HELLO"`
- D) `"hello"`

8. What is the value of `10 % 3` ?

- A) 3
- B) 3.33
- C) 0
- D) 1

9. What does `"automation"[0:4]` return?

- A) `"auto"`
- B) `"autom"`
- C) `"a"`
- D) `"tion"`

10. Which statement correctly converts user input to an integer?

- A) `age = input(int("Enter age: "))`
- B) `age = int.input("Enter age: ")`
- C) `age = int(input("Enter age: "))`
- D) `age = input("Enter age: ").int()`

True / False

#	Statement	Answer
1	In Python, you must declare a variable's type before assigning a value.	False – Python infers types dynamically
2	<code>True</code> and <code>False</code> in Python must be written with a capital first letter.	True
3	The <code>/</code> operator always returns a <code>float</code> result, even when dividing two integers.	True
4	<code>"hello"[1:4]</code> returns <code>"hel"</code> .	False – returns <code>"ell"</code> (index 1,2,3; 4 is excluded)
5	<code>str(42)</code> returns the integer <code>42</code> wrapped in quotes.	True – it returns the string <code>"42"</code>
	String methods like <code>.upper()</code> modify the original string in place.	False – strings are immutable; methods return new

6 #	Statement	strings Answer
7	<code>int(3.9)</code> rounds to 4 .	False — it truncates to 3
8	<code>"abc" * 0</code> produces an error.	False — it produces an empty string ""
9	<code>input()</code> always returns a string, even if the user types a number.	True
10	<code>2 ^ 3</code> computes 8 (2 to the power of 3) in Python.	False — ^ is bitwise XOR; use <code>2 ** 3</code> for exponentiation

Fill in the Blanks

1. To get the number of characters in a string `s` , use `_____ (s)` .
- Answer: `len`
2. The operator `_____` returns the remainder of a division.
- Answer: `%` (modulo)
3. `"Python" [ _____ ]` returns the last character `"n"` .
- Answer: `-1`
4. To convert a string `"99"` to an integer, write `_____ ("99")` .
- Answer: `int`
5. `"hello" _____ "world"` joins two strings to produce `"helloworld"` .
- Answer: `+`
6. To remove leading and trailing whitespace from a string, call the `_____ ()` method.
- Answer: `strip`
7. `_____ division` always produces a `float` result; `_____ division` truncates to an integer.
- Answer: True division (`/`); Floor division (`//`)
8. In Python, the exponentiation operator is written as `_____` .
- Answer: `**`

Interview Questions

- Q1: What are Python's four basic data types covered in this week?**
- A: `str` (text), `int` (whole numbers), `float` (decimal numbers), and `bool` (`True` or `False`). Python infers the type from the assigned value — no explicit type declaration is needed. This is called **dynamic typing**.
- Q2: What is the difference between `/` and `//` in Python?**
- A: `/` is **true division** and always returns a `float` (e.g., `10 / 2 → 5.0`). `//` is **floor division** and returns the largest integer less than or equal to the true result (e.g., `10 // 3 → 3`, `-7 // 2 → -4`). Use `//` when you need a whole-number result, for example when calculating array indices or page counts.
- Q3: Why does Python raise a `TypeError` when you write `"Score: " + 95` ?**
- A: Python is **strongly typed** — it does not implicitly convert types during operations. The `+` operator between two strings performs concatenation, but `95` is an `int`, not a string. To fix it, explicitly convert: `"Score: " + str(95)` . Alternatively, use an f-string: `f"Score: {95}"` , which handles the conversion automatically.
- Q4: What does `input()` return, and why does it matter?**
- A: `input()` always returns a `str` , regardless of what the user types. If the user enters `25` , Python stores the string `"25"` , not the integer `25` . Attempting arithmetic on it without conversion will raise a `TypeError` . Always convert the result: `age = int(input("Enter age: "))` . This is a very common beginner bug.

Q5: What is string slicing and give two practical uses?

A: Slicing extracts a substring using the syntax `s[start:stop:step]`, where `stop` is excluded. Examples of practical use: (1) `filename[-4:]` extracts the last 4 characters (e.g., `.txt` or `.csv`) to check a file extension; (2) `s[::-1]` reverses a string, used in palindrome detection algorithms and data validation routines.

Mini Quiz – Self-Test (No Answers Provided)

Answer these without looking at notes. Check your answers only after you have attempted all six.

1. What is the output of `print(10 % 3)` ?
 2. Write a single line that converts the string `"55"` to an integer and adds `5` to it.
 3. What does `"hello world".split()[1]` return?
 4. What is the difference between `=` and `==` in Python?
 5. Write a slice expression that extracts `"World"` from `"Hello, World!"` .
 6. What happens when you call `int("abc")` ? What error does Python raise?
 7. Write a single expression that produces the string `"ha"` repeated 5 times.
 8. What does `bool(0)` return, and what does `bool("hello")` return?
-

Cross References

- **Concept files:** [Lab_2_1_Variables_and_DataTypes.md], [Lab_2_2_Operators.md], [Lab_2_3_StringOps_TypeConversion.md]
- **Practice:** [Practice_Problems.md]
- **Solutions:** [Solutions.md]
- **Challenge:** [Challenge_Lab.md]
- **Next week:** Week 3 – Control Flow (`if/elif/else`, `while`, `for`, `break`, `continue`)